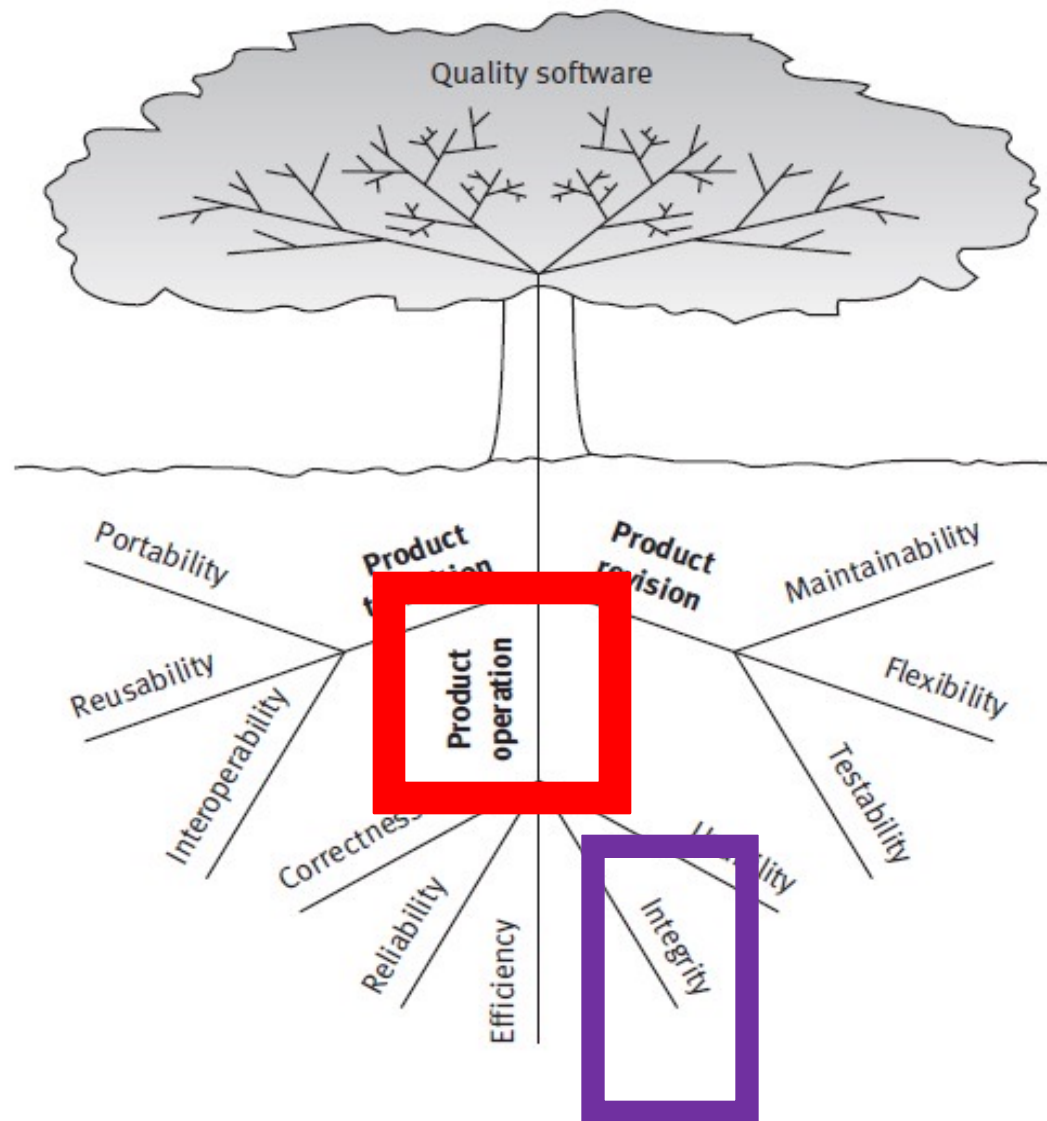
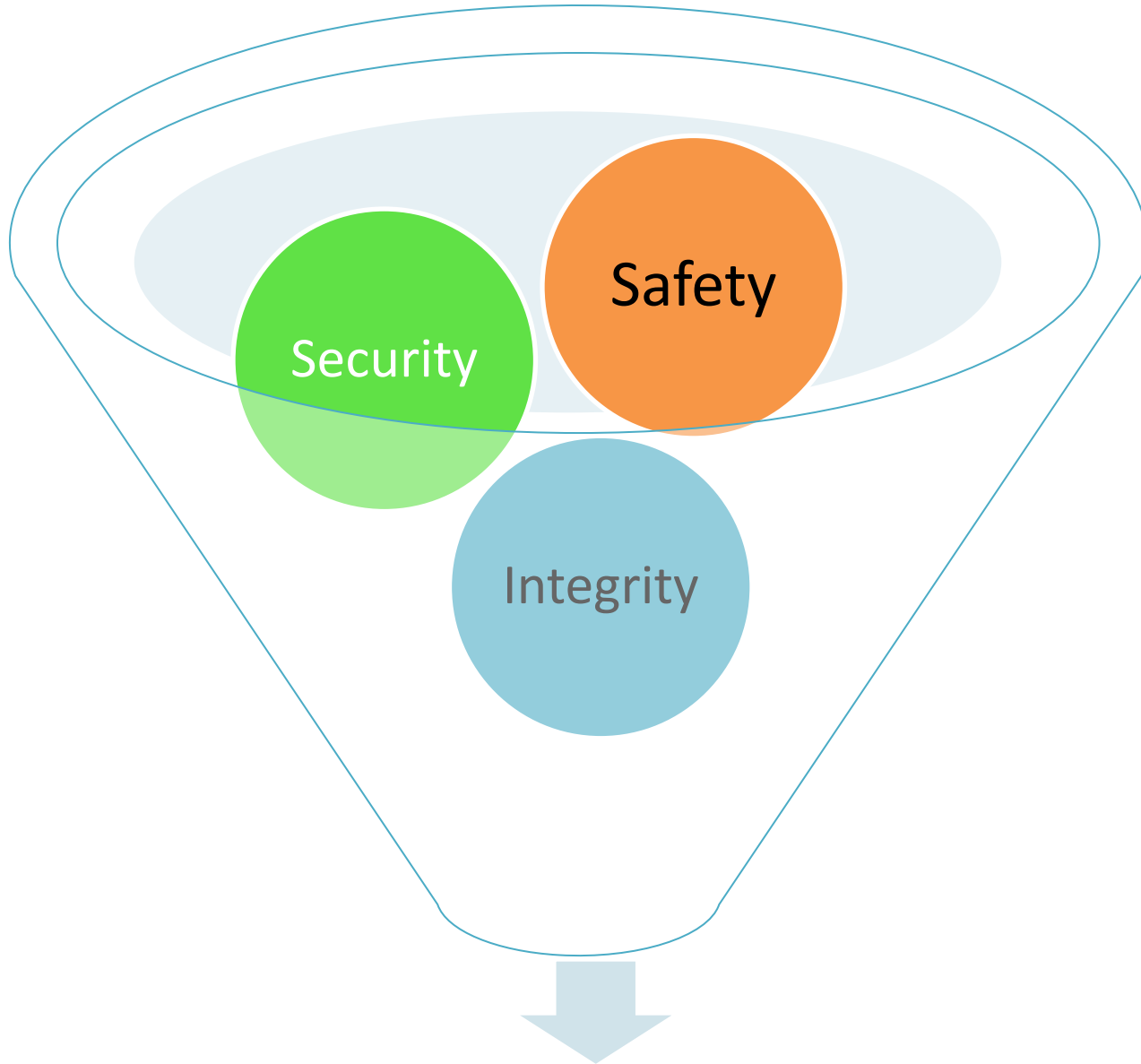


Course 7

Software Quality Factors in detail

Integrity / Security / Safety





Integrity vs. Security vs. Safety

Integrity

- Extent to which access to software or data by unauthorized persons can be controlled [McCall].
- Internal consistency or lack of corruption in electronic data and at the human level
- Prevents unauthorized or improper access & allow proper access

Integrity = ability to withstand attacks to its security

Security

- Protect data and resources contained in and controlled by the software
- Risk for privacy

-Safety requires security: when unauthorized access results in an incident
-Security $\subseteq$ safety – when any unauthorized access is an accident

Safety

- Freedom from software hazards
- Elimination of conditions hazardous to operation as a result of errors in process control software [Deutsch & Willis model]
- Risk for human lives or environment

Integrity [McCall]

- Criteria:
 - Access control
 - Access audit
- Measured in: analysis, design, implementation
- Impact in operation

Access Control

- User access control (ID, password)
- Database access control (authorization table, privacy)
- Memory protection across tasks (mobile app, partitions)

Access Audit

- Support for recording and reporting access
- Support for awareness of access violation

Security

- Remarks:
 - Weakest link
 - Environment dependable
- Approach - Risk analysis:
 - Identify risks
 - Evaluate ways to protect against it
- Detect security vulnerabilities



TOOLS FOR SECURITY VULNERABILITIES

OWASP Top 10 Security Risks:

2021/ 2017 / 2013

OWASP = Open Web
Application Security Project

www.owasp.org

2017

1. Injection
2. Broken Authentication and session management
3. Sensitive Data Exposure (former 6)
4. XML external entities (**new**)
5. Broken access control (**new**)
6. Security misconfiguration (former 8)
7. Cross-Site Scripting
8. Insecure deserialization (**new**)
9. Using components with known vulnerability
10. Insufficient Logging & Monitoring

2021

1. Broken access control (**former 5**)
2. Cryptographic failures (**former 3** Sensitive Data Exposure)
3. Injection (**former 1**)
4. Insure Design (**new**)
5. Security misconfiguration (**former 6**)
6. Vulnerable and outdated components (**former 9** Using components with known vulnerability)
7. Identification and Authentication Failures (**former 2** Broken Authentication and session management)
8. Software and data Integrity Failures (**new**)
9. Security Logging and Monitoring Failures (**former 10** Insufficient Logging & Monitoring)
10. Server-Side Request Forgery – SSRF (**new**)

SonarQube –OWASP:

<https://docs.sonarqube.org/latest/user-guide/security-rules/>

Issues Breakdown by OWASP Category and Severity

Category	Rating	OWASP Vulnerabilities					HotSpots
		!	⬆	⬇	⬆	i	
A1 - Injection	E	12					18
A2 - Broken Authentication	D		6				16
A3 - Sensitive Data Exposure	B				17		14
A4 - XML External Entities (XXE)	E	1					
A5 - Broken Access Control	D		1	1			4
A6 - Security Misconfiguration	E	23	5				4
A7 - Cross-Site Scripting (XSS)	E	1					2
A8 - Insecure Deserialization	A						2
A9 - Using Components with Known Vulnerabilities	A						
A10 - Insufficient Logging & Monitoring	A						
		37	12	1	17	0	60

OWASP

- OWASP Mobile Security Testing Guide & Security Verification Standard
- OWASP SAMM - Software Assurance Maturity Model
- OWASP ASVS - Application Security Verification Standard
- OWASP Web Security Testing Guide
- OWASP GenAI Security project

CERT – Computer Emergency Center – Carnegie Mellon Univ.

<http://www.cert.org/>

- Software security assurance
- Security measurements and analysis
- Frameworks and protocols
- Methods and tools
- Based on risk analysis

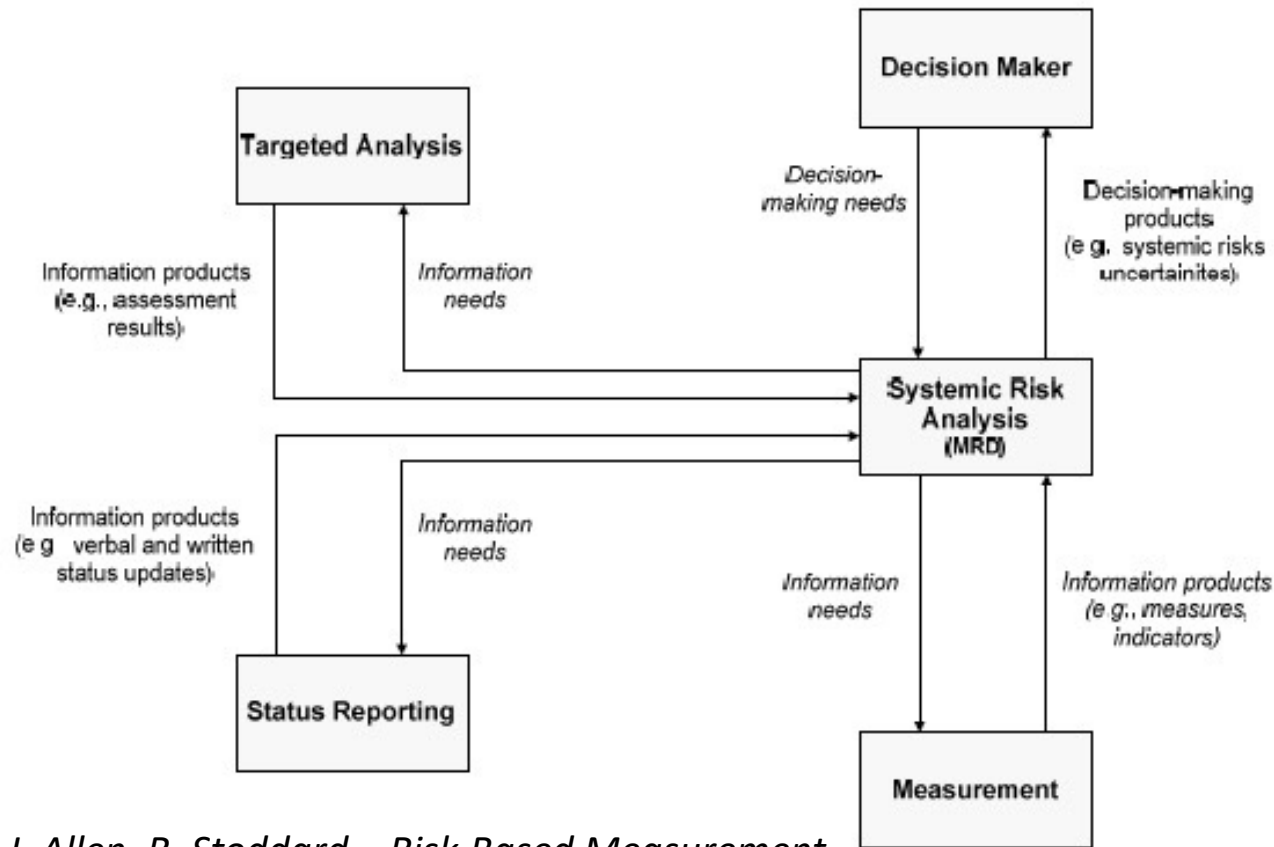


CERT Risk analysis

- 2 approaches:
 1. Tactical
 - risk = probability that an event will lead to a negative consequence or a loss
 - Goal: evaluate systems components for potential failure
 2. Systemic
 - Risk = probability of mission failure

CERT - IMAF

- Integrated Measurement and Analysis Framework
- Application in software security



[C. Alberts, J. Allen, R. Stoddard – *Risk-Based Measurement and Analysis: Application to Software Security*]

- **CWE – Common Weakness Enumeration**
cwe.mitre.org

- Community-developed formal list of common software weaknesses

- Top 25:

- https://cwe.mitre.org/top25/archive/2020/2020_cwe_top25.html

- CWSS: Common Weakness Scoring System
 - CWRAF: Common Weakness Risk Analysis Framework

SonarQube - CWE

Security Assessment
2020 Common Weakness Enumeration (CWE™)
Top 25 Most Dangerous Software Weaknesses (CWE Top 25)

master

CWE Rating



24 CWE Top 25 security issues

232 CWE security issues

43.2% CWE Issues Density

CWE Risk Factor

APPRECIABLE

8.3%

CWE Debt

4d 6h

CWE/SANS Top 25 Issues Breakdown

Rank	ID	Name	CWE/SANS Vulnerabilities					HotSpots
			Rating					
[1]	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	A					2
[2]	CWE-787	Out-of-bounds Write	A					
[3]	CWE-20	Improper Input Validation	A					9
[4]	CWE-125	Out-of-bounds Read	A					
[5]	CWE-119	Improper Restriction of Operations within the Bounds of a Memory Buffer	A					
[6]	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	A					9
[7]	CWE-200	Information Exposure	A					
[8]	CWE-416	Use After Free	A					
[9]	CWE-352	Cross-Site Request Forgery (CSRF)	D	20				2
[10]	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	A					
[11]	CWE-190	Integer Overflow or Wraparound	A					
[12]	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')	A					
[13]	CWE-476	NULL Pointer Dereference	C	3				
[14]	CWE-287	Improper Authentication	A					
[15]	CWE-434	Unrestricted Upload of File with Dangerous Type	A					
[16]	CWE-732	Incorrect Permission Assignment for Critical Resource	A					
[17]	CWE-94	Improper Control of Generation of Code ('Code Injection')	A					
[18]	CWE-522	Insufficiently Protected Credentials	A					
[19]	CWE-611	Improper Restriction of XML External Entity Reference	C	1				
[20]	CWE-798	Use of Hard-coded Credentials	A					11
[21]	CWE-502	Deserialization of Untrusted Data	A					2
[22]	CWE-269	Improper Privilege Management	A					

Others

- IBM, Microsoft, RedHat, Apple,...
- CVSS: Common Vulnerability Scoring System

Case study Microsoft – security checklists

[M. Howard, D. LeBlanc – Writing Secure Code]

- Designer
- Developer
 - General
 - Web & database-specific
 - RPC
 - ActiveX, COM and DCOM
 - Crypto and Secret Management
 - Managed Code
- Tester

Snapshot Designer's Security Checklist

- Person responsible for bug tracking (BugTrack - <http://www.bugtrack.net/>)
- Competitor's vulnerabilities analyzed
- Past vulnerabilities in previous versions analyzed
- Safe-for-scripting controls reviewed
- Sample code reviewed for security issues
- Default install is secure
- Threat models complete for design phase
- Security failures logged for later analysis
- . . .

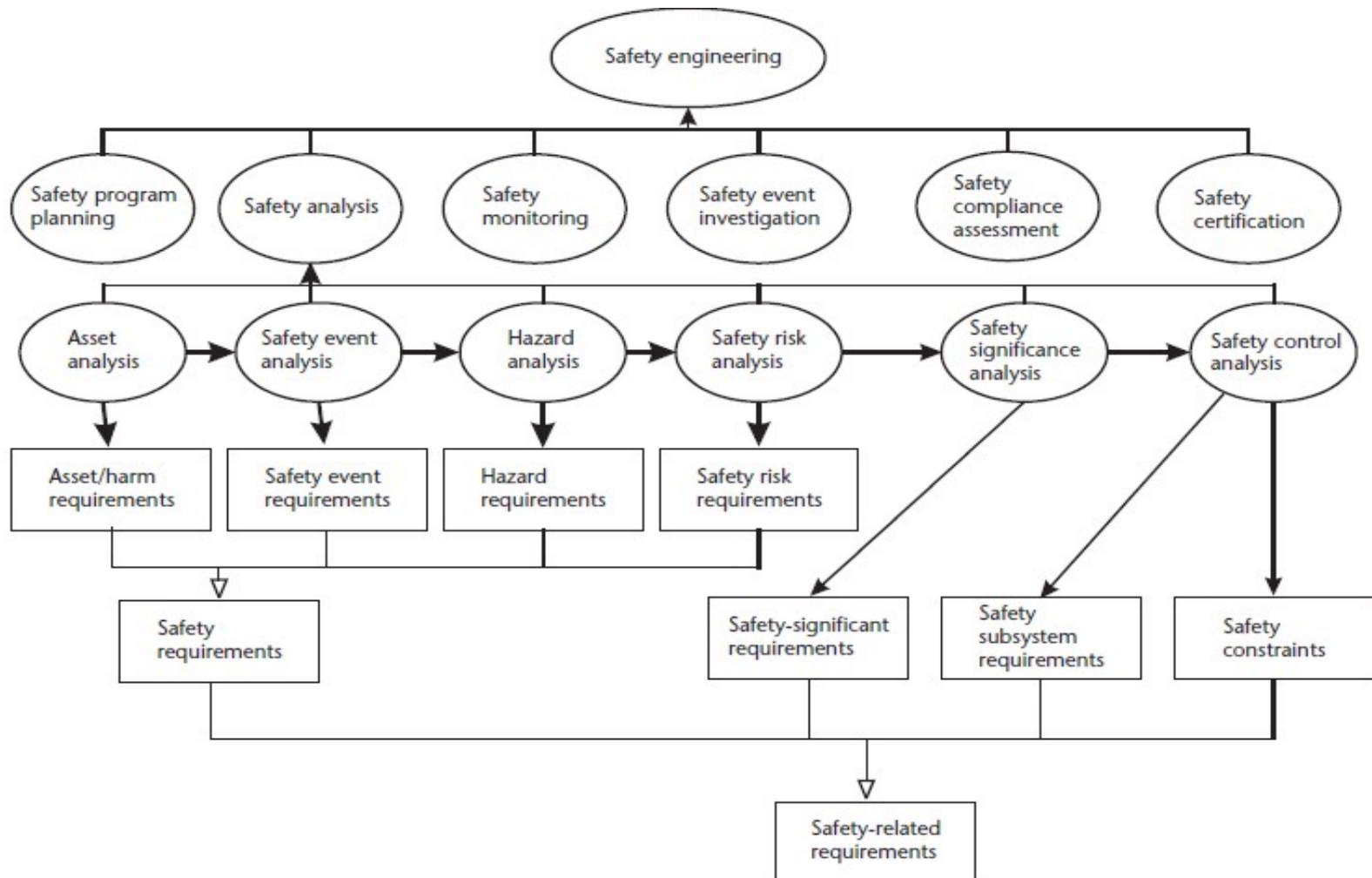
SAFETY

Safety

- Freedom of conditions that can cause death, injury, damage, loss, environmental harm
- Important in safe critical systems
- Harm severity – classification:
 - Catastrophic
 - Critical (severe)
 - Major
 - Minor
 - No effect (negligible)

System Safety Plan

[Firesmith, D., Engineering Safety-Related Requirements for Software-Intensive Systems]



- **1228-1994 - IEEE Standard for Software Safety Plans**
- Software Safety Analysis Procedures – USA, 2014

Safety in embedded systems

- Different standards (depending on industry):
 - **ISO 26262** – ISO for functional safety for automotive
 - **MISRA** – C/C++ standard for **M**otor **I**ndustry **S**oftware **R**elaibility **A**ssoc.
 - **ASPICE**: ISO/IEC 15504 standard (**SPICE** – Software Process Improvement and Capability Determination) for Automotive

MISRA - Motor Industry Software Reliability Association

- C/C++
- **safety, portability, reliability**

Key Objectives:

- Minimize **undefined behavior** in code
- Encourage **predictable execution**
- Reduce **development and maintenance risks**
- **Tools:** Coverity

MISRA Rules Overview:

- **Directive Rules:** High-level guidance
- **Mandatory Rules:** Must be followed with **no deviation**
- **Required Rules:** Can be deviated with **justification**
- **Advisory Rules:** Best practices, not strictly enforced

ASPICE

- Assessment model for automotive software and systems development

Objectives:

- Establish **consistent development processes**
- Enable **capability assessment** through process audits
- Provide a basis for **continuous improvement**
- Rates processes on a scale of **0 to 5**:
 - **Level 1–2**: Performed and Managed
 - **Level 3–5**: Established, Predictable, Innovating

- Acquisition
- System Development
- Software Development
- Supporting Processes
- Management Processes